

WebSocket + Protobuf: Архитектура для веб-игры (Roblox-like)

Стек: React · TypeScript · Babylon.js · ASP.NET Core · Protocol Buffers

Содержание

- 1. Зачем Protobuf в игре
- 2. Архитектура: что через REST, что через WebSocket
- 3. Структура монорепо
- 4. Proto файлы
- 5. Настройка кодогенерации (buf)
- 6. Backend — ASP.NET WebSocket
- 7. Frontend — React + TypeScript
- 8. Интеграция с Babylon.js
- 9. CI/CD — GitHub Actions
- 10. Makefile
- 11. Флоу разработки

1. Зачем Protobuf в игре

Protobuf — бинарный формат сериализации от Google. В сравнении с JSON:

Характеристика	JSON	Protobuf
Читаемость	✔ текст	✗ бинарный
Размер payload	baseline	~3-5× меньше
Скорость парсинга	baseline	~5-10× быстрее
Типизация	zod / TS	встроенная из <code>.proto</code>
Порог входа	низкий	средний

Для игры критично, потому что:

Тип данных	Частота	Protobuf нужен?
Позиции игроков (x, y, z)	~20-60 раз/сек	✅ критично
Анимации / стейт	~10-30 раз/сек	✅ да
Игровые события (прыжок, выстрел)	по событию	✅ да
REST (авторизация, загрузка карты)	редко	❌ JSON достаточно
Чат	редко	❌ JSON достаточно

gRPC не нужен. gRPC — это RPC-фреймворк поверх HTTP/2, он решает другую задачу. Для игры с кастомным WebSocket достаточно чистого бинарного протокола — protobuf идеален сам по себе.

2. Архитектура: что через REST, что через WebSocket

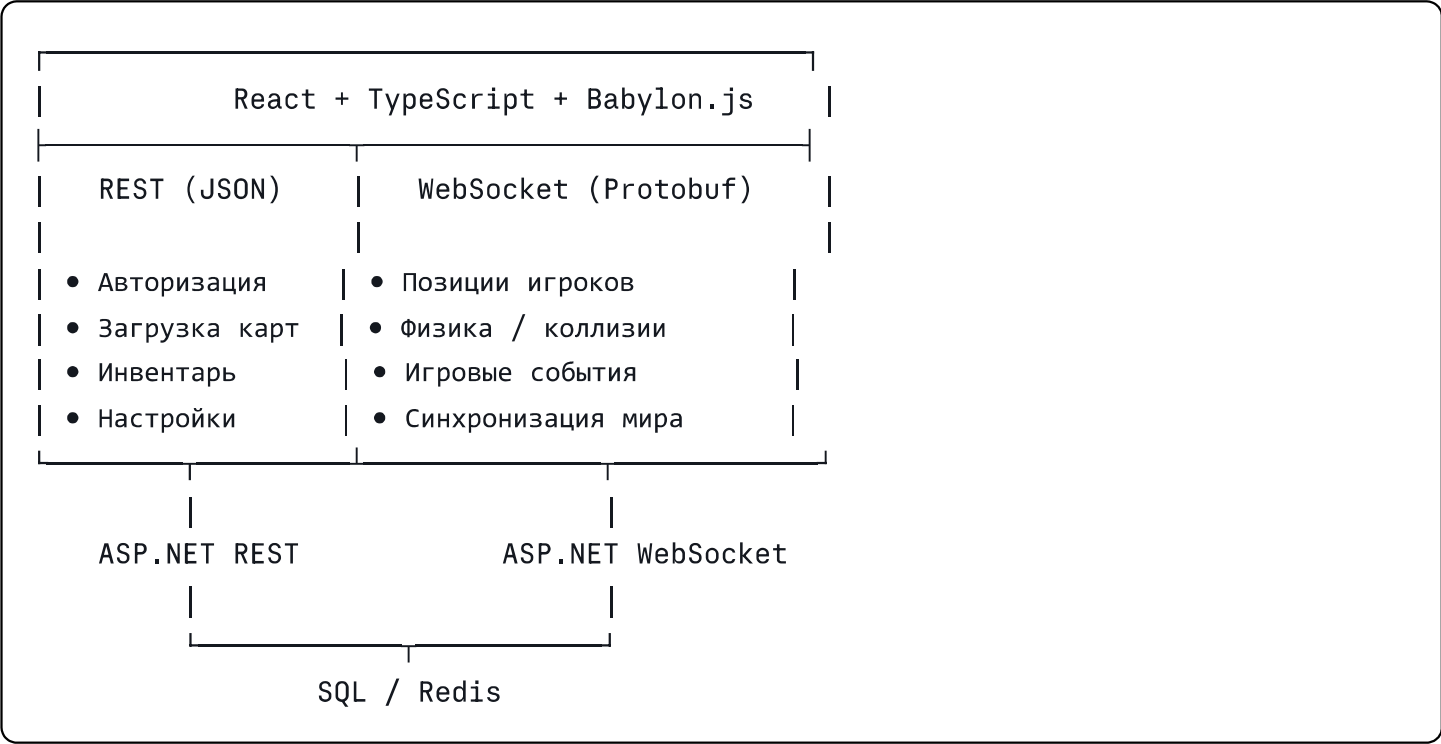
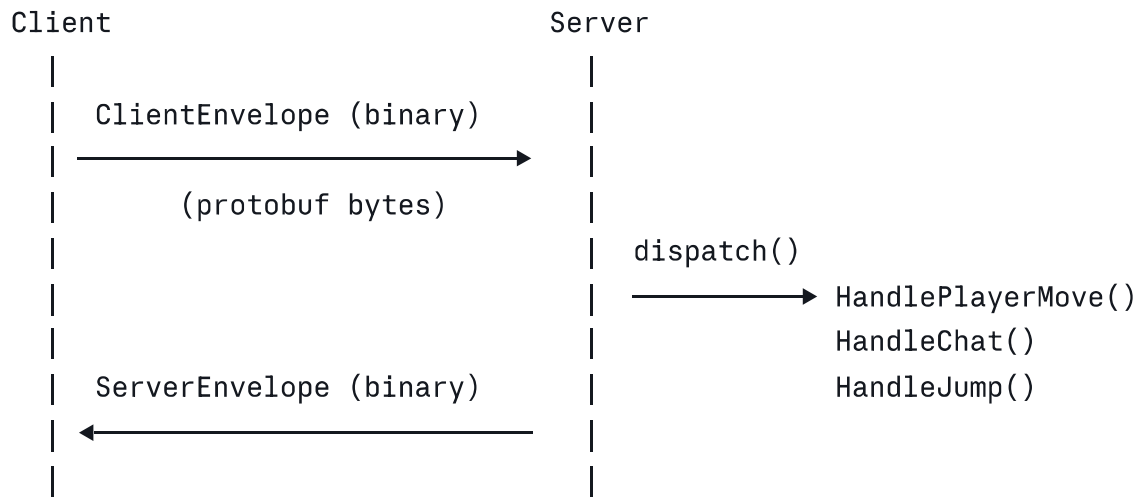


Схема взаимодействия WebSocket:



3. Структура монорепо

```
project/
├─ protos/
│   ├─ buf.yaml
│   ├─ buf.gen.yaml
│   └─ game/
│       ├─ messages.proto
│       ├─ player.proto
│       └─ world.proto
│
├─ backend/                                # ASP.NET Core
│   ├─ backend.csproj
│   ├─ Protos/
│   │   └─ Generated/                    # ← сгенерированные C# классы
│   │   └─ ...
│   └─ ...
│
├─ frontend/                              # React + TypeScript
│   ├─ src/
│   │   └─ gen/                          # ← сгенерированные TS классы
│   ├─ package.json
│   └─ ...
│
├─ Makefile                              # единая точка запуска генерации
├─ .github/
│   └─ workflows/
│       └─ codegen.yml
```

4. Proto файлы

Концепция: Message Envelope

Главная идея — **одно универсальное сообщение-обёртка** с `oneof` внутри. Сервер и клиент всегда шлют `Envelope`, а внутри — конкретный payload.

protobuf

```
// protos/game/messages.proto
syntax = "proto3";
package game;

// Клиент → Сервер
message ClientEnvelope {
    oneof payload {
        PlayerMove    player_move    = 1;
        PlayerJump     player_jump    = 2;
        PlayerShoot    player_shoot   = 3;
        ChatMessage    chat           = 4;
        PingMessage    ping           = 5;
    }
}

// Сервер → Клиент
message ServerEnvelope {
    oneof payload {
        PlayerState    player_state   = 1;
        WorldSnapshot   world_snapshot = 2;
        PlayerDisconnect player_left   = 3;
        ChatMessage     chat           = 4;
        PongMessage     pong           = 5;
        ErrorMessage    error         = 6;
    }
}

// — Shared Types —————

message Vector3 {
    float x = 1;
    float y = 2;
    float z = 3;
}

// — Client → Server Messages —————

message PlayerMove {
    string  player_id = 1;
    Vector3 position   = 2;
    Vector3 rotation   = 3;
    float   speed      = 4;
    uint64  timestamp  = 5; // для client-side prediction
}

message PlayerJump {
```

```
    string player_id = 1;
    Vector3 position = 2;
    uint64 timestamp = 3;
}
```

```
message PlayerShoot {
    string player_id = 1;
    Vector3 origin = 2;
    Vector3 direction = 3;
}
```

```
message PingMessage {
    uint64 client_ts = 1;
}
```

```
// — Server → Client Messages —————
```

```
message PlayerState {
    string player_id = 1;
    Vector3 position = 2;
    Vector3 rotation = 3;
    float health = 4;
    string animation = 5;
}
```

```
message WorldSnapshot {
    repeated PlayerState players = 1;
    uint64 server_ts = 2;
}
```

```
message PlayerDisconnect {
    string player_id = 1;
}
```

```
message ChatMessage {
    string sender = 1;
    string text = 2;
    uint64 sent_at = 3;
}
```

```
message PongMessage {
    uint64 server_ts = 1;
}
```

```
message ErrorMessage {
    string code = 1;
}
```

```
    string message = 2;
}
```

5. Настройка кодогенерации (buf)

buf.yaml

```
yaml
# protos/buf.yaml
version: v2
name: buf.build/yourgame/protos
lint:
  use:
    - DEFAULT
breaking:
  use:
    - FILE
```

buf.gen.yaml

```
yaml
# protos/buf.gen.yaml
version: v2
managed:
  enabled: true
  override:
    - file_option: csharp_namespace
      value: Game.Protos

plugins:
  # TypeScript (frontend)
  - plugin: es
    out: ../frontend/src/gen
    opt:
      - target=ts
      - import_extension=none

  # C# (backend)
  - plugin: protoc-gen-csharp
    out: ../backend/Protos/Generated
    opt:
      - base_namespace=Game.Protos
```

package.json (frontend)

```
json
{
  "scripts": {
    "gen": "cd ../protos && buf generate",
    "gen:clean": "rm -rf src/gen && npm run gen"
  },
  "dependencies": {
    "@bufbuild/protobuf": "^2.0.0"
  },
  "devDependencies": {
    "@bufbuild/buf": "^1.47.0",
    "@bufbuild/protoc-gen-es": "^2.0.0"
  }
}
```

backend.csproj

```
xml
<ItemGroup>
  <PackageReference Include="Google.Protobuf" Version="3.29.0" />
  <PackageReference Include="Grpc.Tools" Version="2.67.0">
    <PrivateAssets>all</PrivateAssets>
  </PackageReference>
</ItemGroup>

<!-- Указываем на сгенерированные файлы -->
<ItemGroup>
  <Protobuf Include="Protos\Generated\**\*.cs"
    GrpcServices="None"
    Generator="MSBuild:Compile" />
</ItemGroup>
```

.gitignore

```
gitignore

# Генерированные файлы — не коммитим, генерируем на месте
frontend/src/gen/
backend/Protos/Generated/

# Buf cache
.buf/
```


Почему не коммитить? Генерированные файлы — это артефакты сборки. Их коммит создаёт конфликты и шум в PR. Генерация занимает секунды.

6. Backend — ASP.NET WebSocket

csharp

```
// GameWebSocketHandler.cs
public class GameWebSocketHandler
{
    private readonly ConcurrentDictionary<string, WebSocket> _players = new();

    public async Task HandleAsync(HttpContext ctx)
    {
        if (!ctx.WebSockets.IsWebSocketRequest) { ctx.Response.StatusCode = 400; ret

        var playerId = ctx.Request.Query["playerId"].ToString();
        using var ws = await ctx.WebSockets.AcceptWebSocketAsync();
        _players[playerId] = ws;

        // Отправляем новому игроку снапшот текущего мира
        await SendWorldSnapshotAsync(playerId);

        await ReceiveLoopAsync(playerId, ws);

        _players.TryRemove(playerId, out _);
        await BroadcastPlayerLeftAsync(playerId);
    }

    private async Task ReceiveLoopAsync(string playerId, WebSocket ws)
    {
        var buffer = new byte[4096];

        while (ws.State == WebSocketState.Open)
        {
            var result = await ws.ReceiveAsync(buffer, CancellationToken.None);

            if (result.MessageType == WebSocketMessageType.Binary)
            {
                var envelope = ClientEnvelope.Parser.ParseFrom(
                    buffer.AsSpan(0, result.Count).ToArray()
                );
                await DispatchAsync(playerId, envelope);
            }
            else if (result.MessageType == WebSocketMessageType.Close)
            {
                break;
            }
        }
    }

    private async Task DispatchAsync(string playerId, ClientEnvelope envelope)
    {
    }
```

```

switch (envelope.PayloadCase)
{
    case ClientEnvelope.PayloadOneofCase.PlayerMove:
        await HandlePlayerMoveAsync(playerId, envelope.PlayerMove);
        break;

    case ClientEnvelope.PayloadOneofCase.PlayerJump:
        await HandlePlayerJumpAsync(playerId, envelope.PlayerJump);
        break;

    case ClientEnvelope.PayloadOneofCase.Chat:
        await BroadcastChatAsync(playerId, envelope.Chat);
        break;

    case ClientEnvelope.PayloadOneofCase.Ping:
        await SendToPlayerAsync(playerId, new ServerEnvelope {
            Pong = new PongMessage {
                ServerTs = (ulong)DateTimeOffset.UtcNow.ToUnixTimeMillisecon
            }
        });
        break;
}
}

private async Task HandlePlayerMoveAsync(string playerId, PlayerMove move)
{
    // TODO: валидация позиции (анти-чит)

    var response = new ServerEnvelope {
        PlayerState = new PlayerState {
            PlayerId = playerId,
            Position = move.Position,
            Rotation = move.Rotation,
        }
    };

    await BroadcastAsync(response, excludePlayerId: playerId);
}

// — helpers —————

private async Task SendToPlayerAsync(string playerId, ServerEnvelope envelope)
{
    if (!_players.TryGetValue(playerId, out var ws)) return;
    var bytes = envelope.ToArray();
    await ws.SendAsync(bytes, WebSocketMessageType.Binary, true, CancellationToken
}

```

```

private async Task BroadcastAsync(ServerEnvelope envelope, string? excludePlayerId)
{
    var bytes = envelope.ToByteArray();
    var tasks = _players
        .Where(p => p.Key != excludePlayerId)
        .Select(p => p.Value.SendAsync(
            bytes, WebSocketMessageType.Binary, true, CancellationToken.None
        ));
    await Task.WhenAll(tasks);
}
}

```

csharp

```

// Program.cs
builder.Services.AddSingleton<GameWebSocketHandler>();
builder.Services.AddControllers();

var app = builder.Build();

app.UseWebSockets();
app.Map("/ws", async ctx =>
    await app.Services
        .GetRequiredService<GameWebSocketHandler>()
        .HandleAsync(ctx));

app.MapControllers(); // REST endpoints остаются как есть
app.Run();

```

7. Frontend — React + TypeScript

useGameNetwork.ts

ts

```
// src/network/useGameNetwork.ts
import { useEffect, useRef, useCallback } from "react";
import { ClientEnvelope, ServerEnvelope } from "@gen/game/messages_pb";

type EventHandler = (envelope: ServerEnvelope) => void;

export function useGameNetwork(playerId: string, onEvent: EventHandler) {
  const wsRef = useRef<WebSocket | null>(null);

  useEffect(() => {
    const ws = new WebSocket(`wss://yourgame.com/ws?playerId=${playerId}`);
    ws.binaryType = "arraybuffer"; // ← обязательно для protobuf

    ws.onmessage = (e: MessageEvent<ArrayBuffer>) => {
      const envelope = ServerEnvelope.fromBinary(new Uint8Array(e.data));
      onEvent(envelope);
    };

    ws.onopen = () => console.log("WS connected");
    ws.onclose = () => console.log("WS disconnected");

    wsRef.current = ws;
    return () => ws.close();
  }, [playerId]);

  const send = useCallback((envelope: ClientEnvelope) => {
    if (wsRef.current?.readyState === WebSocket.OPEN) {
      wsRef.current.send(envelope.toBinary());
    }
  }, []);

  return { send };
}
```

GameEventDispatcher.ts

ts

```
// src/network/GameEventDispatcher.ts
import {
  ServerEnvelope, PlayerState, WorldSnapshot,
  PlayerDisconnect, ChatMessage
} from "@gen/game/messages_pb";

type Handlers = {
  onPlayerState?: (p: PlayerState) => void;
  onWorldSnapshot?: (w: WorldSnapshot) => void;
  onPlayerLeft?: (p: PlayerDisconnect) => void;
  onChat?: (m: ChatMessage) => void;
  onPong?: (serverTs: number) => void;
};

export function dispatchEnvelope(envelope: ServerEnvelope, handlers: Handlers) {
  switch (envelope.payload.case) {
    case "playerState":
      handlers.onPlayerState?.(envelope.payload.value);
      break;
    case "worldSnapshot":
      handlers.onWorldSnapshot?.(envelope.payload.value);
      break;
    case "playerLeft":
      handlers.onPlayerLeft?.(envelope.payload.value);
      break;
    case "chat":
      handlers.onChat?.(envelope.payload.value);
      break;
    case "pong":
      handlers.onPong?.(Number(envelope.payload.value.serverTs));
      break;
  }
}
```

8. Интеграция с Babylon.js

tsx

```

// src/components/GameScene.tsx
import { useEffect, useRef } from "react";
import { Engine, Scene, MeshBuilder, Vector3 } from "@babylonjs/core";
import { useGameNetwork } from "@network/useGameNetwork";
import { dispatchEnvelope } from "@network/GameEventDispatcher";
import { ClientEnvelope, PlayerMove } from "@gen/game/messages_pb";

export function GameScene({ playerId }: { playerId: string }) {
  const canvasRef = useRef<HTMLCanvasElement>(null);
  const meshesRef = useRef<Map<string, BABYLON.Mesh>>(new Map());
  const myPosRef = useRef<Vector3>(Vector3.Zero());

  const { send } = useGameNetwork(playerId, (envelope) => {
    dispatchEnvelope(envelope, {

      onPlayerState: (state) => {
        // Создаём меш если игрок новый, иначе обновляем позицию
        let mesh = meshesRef.current.get(state.playerId);
        if (!mesh) {
          mesh = MeshBuilder.CreateBox(state.playerId, { size: 1 });
          meshesRef.current.set(state.playerId, mesh);
        }
        const p = state.position!;
        mesh.position.set(p.x, p.y, p.z);
      },

      onWorldSnapshot: (snapshot) => {
        // Начальное состояние мира при входе
        snapshot.players.forEach((p) => {
          const mesh = MeshBuilder.CreateBox(p.playerId, { size: 1 });
          const pos = p.position!;
          mesh.position.set(pos.x, pos.y, pos.z);
          meshesRef.current.set(p.playerId, mesh);
        });
      },

      onPlayerLeft: ({ playerId }) => {
        meshesRef.current.get(playerId)?.dispose();
        meshesRef.current.delete(playerId);
      },
    });
  });

  // Отправка своей позиции
  const sendPosition = (position: Vector3) => {
    send(new ClientEnvelope({

```

```

    payload: {
      case: "playerMove",
      value: new PlayerMove({
        playerId,
        position: { x: position.x, y: position.y, z: position.z },
        timestamp: BigInt(Date.now()),
      })
    }
  }));
};

useEffect(() => {
  const engine = new Engine(canvasRef.current!);
  const scene = new Scene(engine);

  let tick = 0;
  scene.onBeforeRenderObservable.add(() => {
    // Отправляем позицию 20 раз/сек (каждые 3 кадра при 60fps)
    if (tick++ % 3 !== 0) return;
    sendPosition(myPosRef.current);
  });

  engine.runRenderLoop(() => scene.render());
  return () => engine.dispose();
}, []);

return <canvas ref={canvasRef} style={{ width: "100%", height: "100%" }} />;
}

```

9. CI/CD — GitHub Actions

yaml


```
# .github/workflows/codegen.yml
name: Codegen Check

on: [pull_request]

jobs:
  check-codegen:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Install buf
        uses: bufbuild/buf-action@v1

      - name: Lint protos
        run: cd protos && buf lint

      - name: Check breaking changes
        run: cd protos && buf breaking --against '.git#branch=main'

      - name: Setup Node
        uses: actions/setup-node@v4
        with: { node-version: 20 }

      - name: Install frontend deps
        run: cd frontend && npm ci

      - name: Generate
        run: make gen

      - name: Check no diff (generated files up to date)
        run: |
          git diff --exit-code frontend/src/gen || \
            (echo "❌ Run 'make gen' and commit the result" && exit 1)
```

CI упадёт, если кто-то изменил `.proto` но не регенерировал файлы.

10. Makefile

makefile

```
# Makefile (корень проекта)

.PHONY: gen gen-clean lint breaking install

# Генерация для всех таргетов
gen:
    cd protos && buf generate
    @echo "✅ TypeScript → frontend/src/gen"
    @echo "✅ C# → backend/Protos/Generated"

# Очистка и регенерация
gen-clean:
    rm -rf frontend/src/gen
    rm -rf backend/Protos/Generated
    $(MAKE) gen

# Проверка .proto файлов
lint:
    cd protos && buf lint

# Проверка breaking changes (полезно перед merge)
breaking:
    cd protos && buf breaking --against '.git#branch=main'

# Первичная установка
install:
    cd frontend && npm install
    npm install -g @bufbuild/buf
```

11. Флоу разработки

1. Редактируешь protos/game/messages.proto

↓

2. make gen

↓

frontend/src/gen/game/ messages_pb.ts ← TypeScript
backend/Protos/Generated/ Messages.cs ← C#

↓

3. TypeScript и C# сразу покажут ошибки
если контракт нарушен

Главный плюс — `.proto` файл как **единый источник правды**: изменил структуру в одном месте, TypeScript и C# сразу покажут ошибки компиляции. Никаких рассинхронов между фронтом и бэком.

Быстрый старт

```
bash

# 1. Установка зависимостей
make install

# 2. Генерация кода
make gen

# 3. Запуск backend
cd backend && dotnet run

# 4. Запуск frontend
cd frontend && npm run dev
```